Which negatives are really negative? Exposure-aware negative weighting and an imputation prior for graph collaborative filtering under exposure bias

Anonymous

Abstract

Graph recommenders learn from logs of items chosen only if shown, so most sampled negatives are unexposed. We propose a weighting and an offset built on the probability Q that an unlogged pair is positive, computed in closed form from an inverse-propensity imputation and an exposure propensity. Negatives are down-weighted by 1 - Q, and the standardised imputation logit is added to the score as a validated offset. Uniform data are used only for validation and testing, except the shipped Coat propensities, estimated from a uniform sample. With Holm-corrected per-user tests, the method has a higher mean than every graph baseline on all four datasets, including a re-implemented AdvInfoNCE. All differences are significant except against LightGCN-pt on KuaiRec ($p_{\text{Holm}} = 0.4409$). It is the best method we ran on Yahoo! R3 (nDCG@5 0.6866 vs 0.6742) and KuaiRand. On KuaiRec it is significantly lower than DR-JL and MRDR in the main run ($p_{\text{Holm}} = 0.0002$ for both), but not across seeds. On Coat it is above iALS in the main run ($p_{\text{Holm}} = 0.0469$), but not across seeds. Training-free propagation remains best on KuaiRec (0.6378). A shuffled-Q control removes the gain. On Coat, with unmatched grids (108 vs 54 configurations), the offset alone recovers most of it.

Keywords: recommender systems, exposure bias, graph collaborative filtering, propensity, doubly robust

1. Introduction

Graph-based recommenders represent users and items as nodes and score a user–item pair by propagating embeddings along the user–item graph. LightGCN (He et al., 2020) is a standard backbone of this family. Such

models are usually trained on implicit feedback logs, in which an interaction is recorded only when the item was shown and the user responded. The logged graph is therefore not the preference graph: it records the exposure policy as much as the user's taste (Marlin and Zemel, 2009; Chen et al., 2023). A model fitted to such a log treats every absence as informative, and the resulting bias is the subject of a large literature on data missing not at random.

The pairwise loss that trains LightGCN, Bayesian personalised ranking (BPR) (Rendle et al., 2009), draws its negatives uniformly from the items that the user has no logged interaction with. Under exposure bias most of these items were never shown to the user, and some of them would have been liked had they been shown. These are false negatives, and they concentrate on rarely exposed items, where the log carries the least information. Matrix factorisation models have addressed this by treating exposure as a latent variable (Liang et al., 2016) or by re-weighting the likelihood with propensities (Saito et al., 2020). Loss-level debiasing with inverse propensity or doubly robust (DR) estimators (Schnabel et al., 2016; Wang et al., 2019b; Guo et al., 2021) is defined over observed or sampled entries. In our experiments a pointwise DR loss on LightGCN (DR-LightGCN) reaches nDCG@5 of 0.5528 on Yahoo! R3, well below BPR training of LightGCN on the same data (0.6708).

This paper starts from the two nuisance functions that a DR estimator already fits: an exposure propensity p_{ui} and an imputed preference $\hat{Y}_{ui}$. Under an independence assumption (Section 3), Bayes' rule applied to them gives the probability that a pair absent from the log is a positive, conditional on its covariates,

$$Q_{ui} = \Pr\big(Y_{ui} = 1 \mid O_{ui}Y_{ui} = 0,\, x_{ui}\big) = \frac{\hat{Y}_{ui}\,(1 - p_{ui})}{1 - p_{ui}\hat{Y}_{ui}}. \tag{1}$$

The quantity Q_{ui} is at most $\hat{Y}_{ui}$, and it is close to $\hat{Y}_{ui}$ when the estimated exposure propensity is small. We use Q in two places inside an otherwise unchanged LightGCN. Each sampled BPR negative is down-weighted by $1-Q_{uj}$, so that likely false negatives contribute less gradient. The standardised logit of the imputation is added to the score as a fixed offset, whose weight is selected on validation data. Both nuisance functions are fitted once on the biased log before training. Training therefore uses no uniformly collected data, except the shipped Coat propensities, which the dataset authors es-

timated from a uniform sample (Section 4.1). We call the resulting model QP-LightGCN (Q-weighted negatives with an imputation prior offset).

The evaluation design follows from a concern that progress claims in top-N recommendation often do not survive a comparison with tuned simple baselines (Ferrari Dacrema et al., 2019; Rendle et al., 2019; Anelli et al., 2023). Some reported debiasing gains rely on protocols that train on uniformly collected data. We use uniform data only for validation and testing (with the Coat exception noted in Section 4.1). The configuration of each split is selected on validation data, the per-user comparisons are corrected with Holm's procedure (Holm, 1979), and the selected configurations are retrained with additional seeds. The results are mixed, and we report them as such. QP-LightGCN has a higher mean than every graph baseline on all four datasets (significant except LightGCN-pt on KuaiRec), and it is the best method among those we ran on Yahoo! R3 and KuaiRand. On Coat it is above iALS in the main run but not significantly different from it across retraining seeds. On KuaiRec it is below DR-JL and MRDR in the main run and not significantly different from them across seeds. On KuaiRec a training-free propagation operator, DRUP, is the strongest method we ran.

DRUP, a training-free walk-corrected propagation operator, applies inverse-propensity and DR edge estimates to multi-hop propagation and corrects the bias that repeated edges add to walks of more than one hop. We keep DRUP as a baseline and as a component, and we report a negative finding about its correction term. At matched configurations the walk correction does not improve accuracy on the real datasets. It improves score values only in a semi-synthetic setting with known propensities. Training-free DR propagation remains the strongest method on KuaiRec, and the correction is not responsible for that result: the same operator without the correction performs equally well.

The contributions of the paper are as follows.

1. *Method.* A closed-form posterior Q_{ui} for pairs that are not logged positives, computed from an inverse-propensity-weighted additive imputation and an exposure propensity (Eq. (1)). Q is used twice in LightGCN: to weight sampled BPR negatives and as a standardised imputation offset. The method adds one selected hyper-parameter, the offset weight, and one lookup per sampled negative.
2. *Evaluation.* A comparison on four public benchmarks with an unbiased test set (Coat, Yahoo! R3, KuaiRand-Pure and KuaiRec), in which

uniform data are used only for validation and testing (Coat's shipped propensities excepted). The comparison uses per-user paired tests with Holm correction within each dataset, and the main configurations are retrained with three additional seeds.

3. *Mechanism.* Ablations that replace Q by the imputation alone, by item popularity, or by a random permutation of its own values. The shuffled control returns to the level of LightGCN, which shows that the weighting requires an informative Q. The relative contribution of weighting and offset depends on the dataset. On Coat, with unmatched grids (108 vs 54 configurations), the offset alone recovers most of the gain.
4. *A negative result on walk correction.* The idempotent correction of DRUP does not improve accuracy on real data at matched configurations, and it improves scores only under semi-synthetic conditions with known propensities.

The rest of the paper is organised as follows. Section 2 reviews the related work and Table 1 positions the method against the main families. Section 3 defines the setting and notation. Section 4 describes the method and the DRUP baseline, Section 5 the datasets, baselines and protocol, and Section 6 the results. Section 7 discusses when the method helps and when it does not, together with its limitations, and Section 8 concludes.

2. Related work

2.1. Graph collaborative filtering

LightGCN (He et al., 2020) removes the feature transformations and non-linearities of earlier graph convolutional recommenders (Wang et al., 2019a), which reduces propagation to a weighted sum of powers of a normalised adjacency matrix. Later variants approximate infinitely many propagation layers (Mao et al., 2021) or add contrastive views of the graph (Wu et al., 2021; Yu et al., 2022). Closed-form and linear models are strong competitors. EASE (Steck, 2019) learns an item–item matrix in closed form, GF-CF (Shen et al., 2021) casts collaborative filtering as graph signal filtering, and BSPM (Choi et al., 2023) uses a related diffusion view. Implicit matrix factorisation with weighted least squares (Hu et al., 2008) remains a competitive baseline, and carefully tuned simple methods often match published graph models (Anelli et al., 2023). A survey of graph-based recommenders is given by Wu

et al. (2023). These models fit or propagate over the logged graph and do not model which pairs were exposed. The present method keeps the propagation of LightGCN and changes the training signal.

2.2. Debiasing under missing-not-at-random feedback

Ratings and implicit feedback are missing not at random (Marlin and Zemel, 2009), and surveys organise the resulting biases and the causal methods that address them (Chen et al., 2023; Gao et al., 2024). Propensity methods re-weight the loss by inverse propensities or self-normalised estimators (Schnabel et al., 2016; Swaminathan and Joachims, 2015; Saito et al., 2020). Steck (Steck, 2010) studied how training and evaluation should account for missingness when a uniformly sampled subset is available. Doubly robust learning adds an imputation model (Wang et al., 2019b), and later variants reduce its bias or variance (Guo et al., 2021; Dai et al., 2022; Li et al., 2023). An information-theoretic counterfactual formulation is given by Wang et al. (2020). Some debiasing methods use a small uniform sample for training, as in AutoDebias (Chen et al., 2021) and CausE (Bonner and Vasile, 2018). Causal approaches to popularity bias use counterfactual reasoning (Wei et al., 2021), confounder adjustment (Zhang et al., 2021) or disentangled representations (Zheng et al., 2021), and later work studies popularity debiasing through the balance of user and item perspectives (Boratto et al., 2021), controllable trade-offs (Okamura et al., 2024) and invariant learning (Bai et al., 2025). These methods correct the loss or the output of a model. The present method applies propensity and DR estimation to the nuisances that define which pairs act as negatives.

2.3. False negatives and negative weighting in implicit feedback

This is the closest group of methods. ExpoMF (Liang et al., 2016) treats exposure as a latent variable and infers an exposure posterior for unclicked items within a matrix factorisation model. Rel-MF (Saito et al., 2020) re-weights observations by propensity, and unbiased pairwise learning (Saito, 2020) applies the same idea to pairwise objectives. Sampling-based methods treat false negatives directly. SRNS (Ding et al., 2020) identifies likely false negatives in the sampling step, and debiased contrastive learning (Chuang et al., 2020) corrects the sampling of negatives that are in fact positives. In contrastive collaborative filtering, the bias-aware margins of the BC loss (Zhang et al., 2022) and the adversarial re-weighting of negatives in AdvInfoNCE (Zhang et al., 2023) change how hard negatives are weighted. The

present weight $1 - Q$ has the posterior form of ExpoMF but is computed in closed form from the nuisances of a DR estimator. It is applied to a graph model with a pairwise loss and is combined with a prior offset. The original formulations of ExpoMF, SRNS and AdvInfoNCE do not use a propensity and an imputation of this kind.

2.4. Debiasing inside graph models

NAVIP (Kim et al., 2022) weights neighbour aggregation by inverse propensities inside a graph neural recommender. r-AdjNorm (Zhao et al., 2022) tunes the normalisation of the adjacency matrix to control popularity amplification, and graph convolutions are analysed as amplifiers of popularity bias by Chen et al. (2024). APDA (Zhou et al., 2023), CAGED (Que et al., 2025) and DPAA (Islam et al., 2026) learn aggregation weights or debiased message passing. These methods change which adjacency is propagated or how neighbours are aggregated. The present method leaves the propagation of LightGCN unchanged. It changes which pairs count as negatives in the loss and adds a prior to the score.

2.5. Score offsets and priors

Adding a fixed offset to a score is a familiar device. In long-tail classification, logit adjustment adds a class prior to the logits (Menon et al., 2021), and in generalised linear models an offset is a known term of the linear predictor. The offset used here is different in one respect: it is a debiased estimate of population-level preference, obtained from an inverse-propensity-weighted imputation, and not a class prior.

2.6. Evaluation protocols and the gap

Progress claims in top-N recommendation depend on the strength of the baselines and on the evaluation protocol (Ferrari Dacrema et al., 2019; Rendle et al., 2019; Anelli et al., 2023). Per-user comparisons of models must account for the overlap between evaluation splits, since standard t-tests treat the splits as independent (Nadeau and Bengio, 2003). Sampled evaluation metrics can also be inconsistent with their exact counterparts (Krichene and Rendle, 2020). Section 5 describes the protocol used here.

Three lines of work meet in this paper but have rarely been combined. Doubly robust and propensity methods (IPS, DR-JL, MRDR) are pointwise and are usually applied to matrix factorisation. Graph recommenders that address exposure or popularity bias (NAVIP, r-AdjNorm, APDA, CAGED)

Table 1: Positioning of the method relative to the main related families. Entries are qualitative. "Acts on" names the part of the model that the correction changes. "Uniform data" indicates whether a uniformly collected sample enters training ("depends": the original propensity estimation can use one). For the methods that we did not run (APDA, CAGED, DPAA) the entries follow their published descriptions. Coat ships its propensities, which were estimated from a uniform sample by the dataset authors; this affects the propensity-based rows on Coat only.

Method family	Graph backbone	Propensity	Imputation	Acts on	Uniform data
LightGCN, SimGCL	yes	no	no	none (BPR or contrastive loss)	no
NAVIP	yes	yes	no	adjacency	no
r-AdjNorm, APDA, CAGED, DPAA	yes	no	no	adjacency or aggregation	no
IPS, SNIPS, DR, DR-JL, MRDR	no (matrix factorisation)	yes	DR only	loss	no
ExpoMF, Rel-MF, UBPR	no	latent or yes	no	loss or exposure posterior	depends
AdvInfoNCE, BC loss	varies	no	no	negatives or margins	no
AutoDebias, CausE	varies	learned weights	no	loss	yes
DRUP (walk correction)	yes	yes	yes	multi-hop adjacency	no
DR adjacency (no correction)	yes	yes	yes	adjacency	no
QP-LightGCN (this paper)	yes	yes	yes	negatives and score	no

change the adjacency or the aggregation rather than the training signal. False-negative-aware sampling (ExpoMF, SRNS, AdvInfoNCE) does not use the propensity and imputation that DR estimation provides. We are not aware of a previous study that uses the DR nuisances to weight the negatives of a graph recommender, and few evaluate graph debiasing on Coat, Yahoo! R3, KuaiRand and KuaiRec together with uniform data held out from training. Some debiasing methods train on uniformly collected data (Chen et al., 2021; Bonner and Vasile, 2018), and several studies report that tuned simple baselines are competitive with published recommenders (Ferrari Dacrema et al., 2019; Rendle et al., 2019; Anelli et al., 2023).

3. Problem setting and notation

Users, items and outcomes.. There are m users $u \in [m]$ and n items $i \in [n]$. Each pair (u, i) has a binary potential outcome $Y_{ui} \in \{0, 1\}$, which records whether user u would like item i if it were shown. Each pair also has an exposure indicator $O_{ui} \in \{0, 1\}$, which records whether the pair was exposed. A pair is a logged positive when $O_{ui}Y_{ui} = 1$. The training data record the exposure indicator O and the logged outcome $R_{ui} = O_{ui}Y_{ui}$. The outcome Y is not recorded on unexposed pairs. The outcome Y is what the recommender should estimate, and the exposure O is the mechanism that makes the log biased.

What O records depends on the platform. On KuaiRec it is an impression chosen by the system. On Coat and Yahoo! R3 it is a rating that the user chose to give, so the "exposure" modelled by the propensity is self-selection by the user rather than the display policy of a platform. All statements below are relative to the mechanism that generates O, and we keep the word exposure for brevity.

Data.. The training data are a biased log $\mathcal{D}_{\text{log}} = \{(u, i, O_{ui}, O_{ui}Y_{ui})\}$ generated by the exposure mechanism. A separate uniformly collected sample $\mathcal{D}_{\text{unif}}$ contains outcomes for randomly exposed pairs. The uniform sample is used only to select hyper-parameters and to evaluate models. It never enters the training objective, and the propensity and imputation models are fitted on the log alone. The one exception is the Coat dataset, whose shipped propensities were estimated by its authors from a uniform sample (Schnabel et al., 2016); Section 7 returns to this point.

For each user u, the set of non-logged pairs is

$$\mathcal{N}_u = \{\, i \in [n] : O_{ui}Y_{ui} = 0 \,\}.$$

This set contains exposed items that the user did not choose and unexposed items. A pairwise sampler draws negatives for a logged positive (u, i) from the items of the catalogue and redraws once when the sampled item is a logged positive. The negatives are therefore approximately uniform on $\mathcal{N}_u$.

Exposure model.. Let x_{ui} denote the covariates of the pair that the nuisance models use, for example user and item exposure rates. The exposure propensity is $p_{ui} = \Pr(O_{ui} = 1 \mid x_{ui})$, and the imputation is $\hat{Y}_{ui} \in [0, 1]$, an estimate of $\Pr(Y_{ui} = 1 \mid x_{ui})$ fitted on the log. Both are functions of the covariates only. The method relies on the following modelling assumption.

Assumption 1 (Exposure independent of the outcome given covariates)**.** *Conditionally on x_{ui}, the exposure O_{ui} and the outcome Y_{ui} are independent.*

Under this assumption the joint distribution of (O_{ui}, Y_{ui}) factorises given x_{ui}, and the posterior Q_{ui} of Eq. (1) follows from Bayes' rule. The quantity $Q_{ui} = \Pr(Y_{ui} = 1 \mid R_{ui} = 0, x_{ui})$ is the probability that a non-logged pair, which is the only kind of pair sampled as a negative, is a positive. It is conditional on the covariates and on $R_{ui} = 0$ only. It does not condition on O_{ui}. Assumption 1 excludes exposure that depends on Y beyond the covariates. It is the condition under which the nuisance models of this paper can be interpreted as posterior quantities, and it is discussed as a limitation in Section 7.

To keep the inverse weights finite, the propensity is clipped from below at a floor $\tau > 0$, and we write $\bar{p}_{ui} = \max(\hat{p}_{ui}, \tau)$ for the clipped estimate. The experiments use $\tau = 0.05$.

Evaluation target.. At test time we rank the items of each user. For user u the candidates are the items that the user never interacted with in the log, and the relevance of a candidate is its outcome Y_{ui} as observed in the uniform test sample. The accuracy measure is a ranking metric, nDCG@K, computed on this uniform sample. The cut-off K depends on the dataset and is given in Section 5. Because these candidates have no logged positive, the evaluation asks about preferences that the log does not reveal directly, which is the quantity that the debiasing methods are meant to recover. Tests and model selection are also described in Section 5.

Scores.. A recommender produces a score s_{ui} for each pair. For a graph model with parameters θ, we write $f_\theta(u, i)$ for its propagated score. The offset added to this score and the weights of the negatives are defined in Section 4. The edge estimates used by the DRUP baseline are defined there as well, since only that baseline needs them.

4. Method

The method keeps LightGCN (He et al., 2020) as the scoring model and changes two things. First, the negatives in the Bayesian personalised ranking loss (Rendle et al., 2009) are weighted by the probability that each sampled unlogged pair is a positive. Second, the score contains a fixed offset built from an imputed outcome. Both quantities come from two nuisance models

Algorithm 1 QP-LightGCN: training on the biased log with Q-weighted negatives and an imputation offset

Require: biased log (O, $O \odot Y$); job grid $\mathcal{J}$ over f_n, β, learning rate, weight decay, width and depth; splits $k = 1, \ldots, K$ with validation parts $\mathcal{V}_k$ and test parts $\mathcal{T}_k$

Ensure: for each split k, the test nDCG of the job, configuration and epoch with the best validation nDCG on $\mathcal{V}_k$

$\hat{p} \leftarrow$ logistic exposure model fitted to O (on Coat, the shipped propensities), truncated below at 0.05 ▷ log only

$\hat{Y} \leftarrow$ additive model fitted to $O \odot Y$ with inverse-propensity weights $O/\hat{p}$, $\lambda = 5$, 20 sweeps, clipped to $[0, 1]$ ▷ log only

$Q_{ui} \leftarrow \hat{Y}_{ui}(1 - \hat{p}_{ui})/(1 - \hat{p}_{ui}\hat{Y}_{ui})$ for all pairs, clipped to $[0, 1]$

$z_{ui} \leftarrow$ logit of $\hat{Y}_{ui}$ (clipped to $[10^{-3}, 1 - 10^{-3}]$), standardised over all pairs

for each job $g \in \mathcal{J}$ **do**

    train LightGCN f_θ with the loss of Eq. (3) and score $f_\theta + \beta_g z$ on the logged positives

    after each epoch, compute nDCG on the validation and test parts of every split (test values are stored and never used for selection)

end for

for each split k **do**

    select the job, configuration and epoch with the best validation nDCG on $\mathcal{V}_k$

    report the test nDCG of the selected model on $\mathcal{T}_k$

end for

that are fitted once on the log before training: an exposure propensity $\hat{p}$ and an imputed outcome $\hat{Y}$. Algorithm 1 summarises the procedure. We call the method QP-LightGCN. Apart from the shipped Coat propensities, which are discussed below, nothing in this section uses uniformly collected data. Those data enter only the selection and evaluation described in Section 5.

4.1. Nuisance models

Exposure propensity.. On Coat we use the propensities shipped with the dataset. On Yahoo! R3, KuaiRand-Pure and KuaiRec we fit a logistic exposure model on the log,

$$\hat{p}_{ui} = \sigma\big(a + b\,\tilde{\ell}_u + c\,\tilde{\ell}_i\big),$$

where $\tilde{\ell}_u$ and $\tilde{\ell}_i$ are standardised logits of the user and item exposure rates (the fraction of items logged for each user, and the fraction of users who logged each item). The coefficients are fitted by Newton's method on all mn pairs of the log. Every propensity is truncated below at 0.05, including the shipped ones. The model uses the exposure matrix O only.

Imputed outcome.. We fit the additive model $\hat{Y}_{ui} = \min\{1, \max\{0, \mu + b_u + b_i\}\}$ to the logged outcomes $O \odot Y$, weighting each observed pair by $1/\hat{p}_{ui}$. The parameters are updated in closed form by 20 alternating sweeps of ridge regression with penalty $\lambda = 5$. The fit uses the log only, and it is not cross-fitted, so the same log supplies $\hat{Y}$ and the training data. The truncation level, the penalty and the number of sweeps are fixed in the code. They are not selected on the validation or test data.

On Coat the shipped propensities were estimated by the dataset authors from the uniform ratings (Schnabel et al., 2016). These are the same uniformly collected ratings that serve for validation and testing. Coat is therefore the one dataset where uniform data reach a training nuisance, indirectly. We report Coat with this caveat, and it is listed among the limitations. The same shipped propensities enter every propensity-based baseline on Coat (DR-LightGCN, NAVIP, DR-JL, MRDR, DR adjacency and DRUP), so the Coat comparison shares this input.

4.2. Posterior of an unlogged pair

Under the notation of Section 3, a pair with $O_{ui}Y_{ui} = 0$ is one whose outcome is not revealed by the log. It is either unexposed or exposed with a negative outcome. BPR draws its negatives from this set. The probability that such a pair is a positive is

$$Q_{ui} = \Pr\big(Y_{ui} = 1 \mid O_{ui}Y_{ui} = 0,\, x_{ui}\big) = \frac{\hat{Y}_{ui}\,(1 - \hat{p}_{ui})}{1 - \hat{p}_{ui}\,\hat{Y}_{ui}}. \tag{2}$$

Here $\hat{p}$ denotes the truncated propensity $\bar{p}$ that the code uses. The conditioning in Q is on the covariates x_{ui} and on $O_{ui}Y_{ui} = 0$ only, not on O_{ui}. The derivation assumes that, given the covariates x_{ui} on which the propensity depends, exposure is independent of the outcome, and that $\hat{Y}_{ui}$ estimates $\Pr(Y_{ui} = 1 \mid x_{ui})$. Then $\Pr(Y_{ui} = 1, O_{ui} = 0) = \hat{Y}_{ui}(1 - p_{ui})$ and $\Pr(O_{ui}Y_{ui} = 0) = 1 - p_{ui}\hat{Y}_{ui}$. The closed form therefore does not cover

an exposure mechanism that depends on the outcome beyond those covariates; Section 3 states this as Assumption 1. We treat the independence as a modelling assumption and do not test it.

Three properties follow. First, $0 \le Q_{ui} \le \hat{Y}_{ui}$. Second, $Q_{ui} \to \hat{Y}_{ui}$ as $\hat{p}_{ui} \to 0$, so a pair that is almost never exposed keeps its prior. Third, $Q_{ui} \to 0$ as $\hat{p}_{ui} \to 1$ when $\hat{Y}_{ui} < 1$, so a pair with an estimated exposure propensity near one and no logged positive keeps its full weight as a negative. The quantity has the same form as the exposure posterior of ExpoMF (Liang et al., 2016). Here it is computed in closed form from the two nuisances and needs no latent-variable model. In the code, Q is clipped to $[0, 1]$ and the denominator is bounded below by 10^{-6}. Q is used as a weight. We do not claim that the resulting training loss is unbiased.

4.3. Q-weighted BPR

Training uses only the logged positives. For each positive pair (u, i) one negative item j is drawn uniformly from all items. If (u, j) is itself a logged positive, j is drawn once more. For a mini-batch B of such triples the loss is

$$\begin{aligned} \mathcal{L} = &- \frac{\sum_{(u,i,j)\in B} w_{uj}\, \log \sigma\big(s_{ui} - s_{uj}\big)}{\sum_{(u,i,j)\in B} w_{uj}} \\ &+ \lambda \frac{1}{|B|} \sum_{(u,i,j)\in B} \big(\|e_u\|^2 + \|e_i\|^2 + \|e_j\|^2\big), \\ w_{uj} = &\max\big\{1 - f_n\, Q_{uj},\ 10^{-3}\big\}. \end{aligned} \tag{3}$$

where e_u, e_i and e_j are the propagated embeddings and λ is the weight decay. The weight acts on the negative only. The loss is normalised by the total weight of the batch, so its scale does not depend on how many negatives are down-weighted. The factor f_n equals 1 in every final job except the Coat job without offset, where $f_n \in \{0.5, 1\}$ is searched. With $f_n = 1$ and Q_{uj} close to one, the sampled negative contributes almost nothing.

The score s_{ui} is defined in Section 4.4. The encoder f_θ is LightGCN with symmetric normalisation of the unweighted graph of logged positives and a mean over the layer outputs. The depth and width are tuned as described in Section 5. Adam is used for optimisation, and training stops early on each split's validation score.

4.4. Imputation offset

Let $z_{ui} = (\ell_{ui} - \bar{\ell})/s_\ell$, where $\ell_{ui} = \text{logit}\, \hat{Y}^c_{ui}$, $\hat{Y}^c$ is $\hat{Y}$ clipped to $[10^{-3}, 1 - 10^{-3}]$, and $\bar{\ell}$ and s_ℓ are the mean and standard deviation of ℓ over all mn pairs. The score is

$$s_{ui} = f_\theta(u, i) + \beta\, z_{ui}, \qquad \beta \in \{0,\, 0.5,\, 1,\, 2\}. \tag{4}$$

The offset is used in training and at inference. It is added to the sampled negative as well, so it acts on the margin between a positive and its negative: $s_{ui} - s_{uj} = f_\theta(u, i) - f_\theta(u, j) + \beta(z_{ui} - z_{uj})$. The offset is fixed, so the training loss cannot change it. The model therefore learns a residual on top of the imputed preference. Logit adjustment for long-tailed classification (Menon et al., 2021) also adds a fixed prior to the logits, but here the prior is an inverse-propensity estimate of a population-level preference rather than a class frequency. The value $\beta = 0$ removes the offset. Each split selects β on its validation part, as one of the jobs in the grid.

4.5. Procedure and cost

Algorithm 1 lists the steps. Steps 1 to 4 use the log only. The uniform data are read only in the evaluation inside the training loop and in the selection that follows, and they never enter the loss. The propensity, the imputation, Q and z are dense $m \times n$ arrays, so memory grows with mn. The largest catalogue used here is KuaiRec, with 7,176 users and 10,728 items. Each training step adds one lookup of Q per sampled negative and one lookup of z per positive and per negative. LightGCN itself is unchanged. For larger catalogues Q and z could be computed on the fly from the imputation parameters and the propensity model. We have not implemented this. We do not report run times, because the result files do not contain a controlled timing comparison.

4.6. Training-free DR propagation and the walk correction

DRUP is a training-free walk-corrected propagation operator. In this paper it plays two roles: as a baseline, and as a secondary component whose correction we test separately. From the log it forms two edge estimates for every pair. The first is the inverse-propensity estimate $W^{\text{IPS}} = OY/\bar{p}$. The second is the doubly robust estimate $W^{\text{DR}} = \hat{Y} + O(Y - \hat{Y})/\bar{p}$, where $\bar{p}$ is the truncated propensity (Dudík et al., 2011). The operator propagates these estimates over three hops. A walk that reuses an edge contributes

Table 2: Datasets. "Log": the biased training data, with its density. "Test": the unbiased data. "Candidates": unbiased test items per user that are ranked (items never logged for that user). "Evaluated users": users with at least one positive candidate in the full test set.

Dataset	Users / items	Log (density)	O means	Test	Candidates	Evaluated users
Coat	290 / 300	6,960 (8.0%)	Self-rated	16 random ratings per user	14.7	225
Yahoo! R3	15,400 / 1,000	311,704 (2.0%)	Self-rated	10 random songs, 5,400 users	10	2,315
KuaiRand	10,000 / 7,583	511,863 (0.68%)	Feed impression	random impressions	43.6	8,347
KuaiRec	7,176 / 10,728	10.3M (13.4%)	Impression	fully observed 1,411 × 3,327	3,314	1,411

a biased product, and DRUP's idempotent correction removes the excess that such walks add. The score combines the one-hop and the corrected three-hop terms with a weight that is selected on validation data. Under fixed nuisances, the construction is designed to make the three-hop operator unbiased for propagation over the fully exposed graph. The argument is given in the supplement.

The correction does not improve accuracy on real data in our experiments. At matched configurations, the corrected operator is lower on Coat and close to the uncorrected one on Yahoo! R3 (Section 6). Its gain appears only in a semi-synthetic setting with true propensities and fixed degree weights, and in one of the tested settings it reverses at large walk weights. We therefore present the correction as a secondary, training-free component. The strong accuracy of training-free DR propagation on KuaiRec is reported as a property of the propagation, not of the correction. QP-LightGCN does not use DRUP for training.

5. Experimental setup

5.1. Datasets

We use four public benchmarks. Each pairs a biased log with a test set collected without the selection of the log, so that a debiasing method can be measured against unbiased outcomes. Table 2 gives the sizes, which we checked against the data loaders in the repository.

Coat (Schnabel et al., 2016) has 290 users and 300 items. The log holds 6,960 self-selected ratings (8.0% of the matrix), of which 1,905 are ratings of 4 or more. The test set holds 16 ratings per user on uniformly random items. Candidates are the items a user did not rate in the log, 14.7 per user on average. The dataset ships propensities, which its authors estimated from a uniform sample; the propensity-based methods use them on Coat

(Section 4.1). *Yahoo! R3* (Marlin and Zemel, 2009) has 15,400 users and 1,000 songs. The log holds 311,704 self-selected ratings (2.0% dense). The test set holds 54,000 ratings of ten random songs for 5,400 users, in the version distributed with (Chen et al., 2021). A random 5% of these users (270, fixed seed) is excluded from validation and testing. The remaining 5,130 test users have ten candidates each, and 2,315 of them have a positive candidate. *KuaiRand-Pure* (Gao et al., 2022b) is the only benchmark whose log records platform exposure: an impression is an item shown in the standard feed, and the outcome is a click. We keep 10,000 users who also received randomly exposed videos, and all 7,583 videos. The log holds 511,863 impressions (0.68% dense), 47% of them clicks. The test set consists of the random impressions of the same period, restricted to videos the user was not shown in the standard log. Users have 43.6 candidates on average, and 8,347 of them have a positive candidate. *KuaiRec* (Gao et al., 2022a) provides a dense log of 10.3 million views by 7,176 users of 10,728 items (13.4% dense). Its test set is a fully observed matrix for 1,411 users and 3,327 items. Each user is ranked over the test items that the user never viewed in the log, 3,314 on average.

Positive outcomes are ratings of 4 or more (Coat, Yahoo! R3), a click (KuaiRand) and a mean watch ratio of at least 2 (KuaiRec). On Coat and Yahoo! R3, "exposure" means that the user chose to rate the item, so the propensity models self-selection and not a platform's allocation. The candidate sets of these two datasets hold about ten to fifteen items, which limits the range that nDCG can span on them. The "evaluated users" column of Table 2 counts every user with a positive candidate. The test-user counts in Section 5.4 are slightly lower for Yahoo! R3, KuaiRand and KuaiRec, because a user can fall in the validation part of every split and then has no test entry.

5.2. Baselines

Every method is run by us on the same splits and data preparation. No number is taken from the literature. Graph models are LightGCN (He et al., 2020), r-AdjNorm (Zhao et al., 2022), NAVIP (Kim et al., 2022), SimGCL (Yu et al., 2022), DR-LightGCN and LightGCN-pt. Matrix models are iALS (Hu et al., 2008), BPR-MF, MACR (Wei et al., 2021), DR-JL (Wang et al., 2019b) and MRDR (Guo et al., 2021). AdvInfoNCE is the contrastive baseline (Section 5.3). The training-free operators are DRUP and DR adjacency, which is DRUP without the walk correction. On Coat,

the propensity-based baselines (DR-LightGCN, NAVIP, DR-JL, MRDR, DR adjacency and DRUP) use the same shipped propensities as the method.

LightGCN is trained with BPR on the logged positives and uniform negatives. It is the loss of Section 4.3 with unit weights and no offset. LightGCN-pt is LightGCN trained with a pointwise cross-entropy on all logged pairs (KuaiRec only). r-AdjNorm searches the normalisation exponent $r \in \{0.3, 0.7\}$ on top of the LightGCN grid. NAVIP trains LightGCN with BPR on a neighbourhood aggregation in which each logged edge is weighted by the inverse of its estimated propensity. DR-LightGCN uses the pointwise doubly robust loss with the propensity and imputation of QP-LightGCN, and pairs drawn uniformly from all pairs. SimGCL adds uniform noise to the propagated embeddings and an InfoNCE term between two noisy views, with weight in $\{0.1, 0.5\}$. iALS puts confidence $1 + \alpha$ on logged positives and 1 on the other pairs, with $\alpha \in \{3, 10, 30\}$ and regularisation in $\{1, 10, 100, 1000\}$. BPR-MF is matrix factorisation trained with BPR. Our MACR baseline is a MACR-style model on a matrix factorisation backbone, with user and item branches and popularity removed at inference ($c \in \{0.1, 0.3\}$); it is not the original code. DR-JL and MRDR fit matrix factorisation with the joint imputation of their losses.

The final comparison contains the following methods. Coat: LightGCN, r-AdjNorm, NAVIP, DR-LightGCN, iALS, SimGCL, MACR and AdvInfoNCE. Yahoo! R3: LightGCN, r-AdjNorm, NAVIP, iALS, SimGCL and AdvInfoNCE. KuaiRand: LightGCN, r-AdjNorm, NAVIP, iALS, BPR-MF, SimGCL and AdvInfoNCE. KuaiRec: DR-JL, MRDR, LightGCN-pt, DR-LightGCN, NAVIP, iALS, LightGCN, r-AdjNorm and AdvInfoNCE. These are the Holm families of Section 5.4. DRUP, DR adjacency, and DR-JL and MRDR on Coat and Yahoo! R3 are reported without a test, because they are outside the families.

We did not run StableDR (Li et al., 2023), DICE (Zheng et al., 2021), CausE (Bonner and Vasile, 2018) or AutoDebias (Chen et al., 2021). CausE and AutoDebias need uniformly collected data in training, which our protocol reserves for validation and testing. For StableDR and DICE, the DR and popularity-aware baselines we ran are the closest counterparts. We did not compare with LLM-based recommenders. None of the four benchmarks provides item text together with a random-exposure test set, so such a comparison would test a different hypothesis.

5.3. AdvInfoNCE re-implementation

AdvInfoNCE (Zhang et al., 2023) is an adversarial contrastive loss. We re-implemented it on the same LightGCN encoder, and our numbers are for this re-implementation, not for the original paper. The main simplifications are these.

- The score is the cosine similarity of the propagated embeddings divided by a temperature τ. Each positive is contrasted with 64 sampled negatives.
- The adversary is a pair of embedding tables (users and items, dimension 64) trained with Adam to maximise the contrastive loss, with learning rate 10^{-2} on Coat, 10^{-3} on Yahoo! R3 and KuaiRand, and $5 \cdot 10^{-4}$ on KuaiRec. Only this embedding-based adversary is implemented.
- The adversary budget of the original formulation is replaced by a number of adversarial rounds $\eta \in \{4, 12\}$. A round runs after each epoch from the second epoch on and lasts at least one pass over the positives and 100 Adam steps. The adversary's KL divergence is logged but not projected onto a ball.

The grids differ in size. Coat has 216 configurations: three learning rates, three weight decays, widths $\{32, 64\}$, depths $\{1, 2, 3\}$, $\tau \in \{0.2, 0.75\}$ and $\eta \in \{4, 12\}$. Yahoo! R3 has 16 configurations with $\tau \in \{0.1, 0.28\}$, $\eta \in \{4, 12\}$, learning rates $\{3 \cdot 10^{-3}, 10^{-3}\}$, weight decays $\{10^{-5}, 10^{-4}\}$, width 128 and depth 2. KuaiRand and KuaiRec have 8 each: KuaiRand uses $\tau \in \{0.1, 0.28\}$, learning rates $\{10^{-2}, 3 \cdot 10^{-3}\}$, weight decay 10^{-4}, width 64 and depth 2; KuaiRec uses $\tau \in \{0.2, 2.0\}$, learning rates $\{10^{-2}, 3 \cdot 10^{-3}\}$, weight decay 10^{-5}, width 64 and depth 2; both use $\eta \in \{4, 12\}$. A larger grid gives a method more chances to fit noise in the validation data, so the Coat grid favours AdvInfoNCE and the smaller grids work against it.

5.4. Protocol

Splits.. The uniformly collected data are split into a validation part and a test part. On Coat the split is made within each user: 30% of each user's candidates go to validation and the rest to test. On Yahoo! R3, KuaiRand and KuaiRec the split is made over users: 30% of the test users go to validation, and no user is in both parts of a split. Coat uses ten random splits (seeds 0 to 9) and the other datasets use five (seeds 0 to 4).

Training and selection.. Each configuration is trained once on the log, and the same run serves every split. After each epoch the model is scored on the validation and test parts of every split. Each split keeps its own best (configuration, epoch) by validation nDCG and stops when its validation score has not improved for the patience period: five epochs on Coat and Yahoo! R3, three on KuaiRand and KuaiRec. A run ends when every split has stopped or the epoch limit is reached. Test scores are stored for every epoch, but they never influence the choice. All methods on a dataset share the epoch limit, batch size and patience, except AdvInfoNCE on KuaiRec, which uses batch size 4,096 instead of 32,768.

Metrics.. We report nDCG@5 on Coat and Yahoo! R3, nDCG@10 on KuaiRand and nDCG@20 on KuaiRec. Each user's candidates are ranked within the part being scored, and users with no positive candidate in that part are skipped. We rank all candidates instead of sampling negatives, because sampled metrics are not consistent with the full ranking metric (Krichene and Rendle, 2020). We report the mean and the standard deviation over splits (with ddof = 0). Evaluation uses only the uniformly collected test data, following the recommendation of Steck (2010) to evaluate on data that are not missing not at random.

Tests.. Differences are tested at the level of users. For each user we average the user's test nDCG over the splits in which the user is a test user. For a method and a baseline we take the per-user differences, apply a one-sample t-test, and report the 95% confidence interval as a half-width. Within each dataset the p-values of the family are adjusted by the Holm procedure (Holm, 1979). A difference is called significant only if the adjusted p is below 0.05. Differences are reported as the mean per-user difference times 10^3, with the half-width in brackets. This is not the difference of the two table means. The Holm families have 8 tests on Coat, 6 on Yahoo! R3, 7 on KuaiRand and 9 on KuaiRec. The number of users with a test entry in the final method's files is 225, 2,310, 8,321 and 1,410, respectively.

Splits overlap, so per-user values are not independent across splits, and paired tests over overlapping splits can be anti-conservative (Nadeau and Bengio, 2003). The tests treat the training log, the fitted nuisances and the trained models as fixed. We do not report a corrected resampled test for the final method.

5.5. Seed robustness

Each split's selected configuration is retrained with seeds 1, 2 and 3. Each split re-selects its epoch on its own validation part, and the configuration is not changed. The retrained final method is compared with the two highest-scoring baselines of the main run for each dataset, where "highest-scoring" means highest main-run test mean. The family of two is therefore chosen on the test data, and its tests are descriptive: iALS and DR-LightGCN on Coat, r-AdjNorm and LightGCN on Yahoo! R3 and KuaiRand, and DR-JL and MRDR on KuaiRec. The Holm correction runs over those two tests. The two baselines are retrained in the same way: for each split, the configuration selected in their main-run files is retrained with seeds 1 to 3, and the epoch is re-selected on validation data, as for the final method. Their seed-0 values are the main-run values. Some main-run files also contain a re-run of split 0 with seeds 100 to 102, stored in a separate field. These runs are not used in any table.

5.6. Tuning grids and budgets

The final method is the union of several jobs, one per group of β values and learning rate (and, on Coat, one per value of f_n in the offset-free job). Each split keeps the job with the best validation score (Section 4.4).

- **Coat.** Learning rate $\{10^{-2}, 3 \cdot 10^{-3}, 10^{-3}\}$, weight decay $\{10^{-5}, 10^{-4}, 10^{-3}\}$, width $\{32, 64\}$, depth $\{1, 2, 3\}$, at most 100 epochs and batch size 256. The three jobs (offset $\beta = 0$ with $f_n \in \{0.5, 1\}$; $\beta \in \{0.5, 1\}$; $\beta \in \{1, 2\}$) have 108 configurations each, 324 in total.

- **Yahoo! R3.** Learning rate $\{10^{-2}, 3 \cdot 10^{-3}, 10^{-3}\}$ for the $\beta = 0$ jobs and $3 \cdot 10^{-3}$ for the offset jobs, weight decay $\{10^{-5}, 10^{-4}\}$, width $\{64, 128\}$, depth $\{1, 2, 3\}$, at most 60 epochs and batch size 1,024. The five jobs have 84 configurations in total.

- **KuaiRand.** Learning rate $\{10^{-2}, 3 \cdot 10^{-3}, 10^{-3}\}$, weight decay $\{10^{-5}, 10^{-4}\}$, width 64, depth 2, at most 40 epochs and batch size 8,192. The three jobs have 30 configurations in total.

- **KuaiRec.** Weight decay $\{10^{-5}, 10^{-4}\}$, width $\{64, 128\}$, depth $\{1, 2, 3\}$, at most 30 epochs and batch size 32,768. Learning rates are $\{10^{-2}, 3 \cdot 10^{-3}\}$ for the $\beta = 0$ jobs, $\{10^{-2}, 3 \cdot 10^{-3}, 10^{-3}\}$ for $\beta \in \{1, 2\}$, and $3 \cdot 10^{-3}$ for $\beta \in \{0.5, 1\}$. The six jobs have 120 configurations in total.

The cap of 2,000,000 pairs per epoch never binds, because the log holds 779,468 positives.

The baselines use the same learning-rate and weight-decay values and their own parameters from Section 5.2. The number of configurations searched for each baseline is in the "cfgs" column of the main results table. The baseline counts range from 6 to 216 (AdvInfoNCE on Coat), and the full grid of each baseline will be listed in the supplement. The unequal budgets are a limitation of the comparison, since selection over more configurations can favour the method.

6. Results

All test numbers are nDCG on uniformly exposed test data. Validation and test splits are drawn from the uniform data; the training log is the only data used to fit the nuisances and the model. For each dataset and split, the final method (QP-LightGCN) is the validated combination of its jobs: each split keeps the job, offset weight and epoch with the best validation score. Table 3 reports the main run (seed 0), Table 5 the retraining with seeds 1–3, and the remaining tables the ablations and the walk-correction analysis. Per-user paired tests use the users who appear in the test sets of both methods: 225 on Coat, 2,310 on Yahoo! R3, 8,321 on KuaiRand and 1,410 on KuaiRec.

6.1. Main comparison

Table 3 and Figure 1 give the main comparison.

1. **Graph baselines.** On all four datasets the final method has a higher mean than every graph baseline we ran: LightGCN, r-AdjNorm (Zhao et al., 2022), NAVIP (Kim et al., 2022), SimGCL (Yu et al., 2022) and the AdvInfoNCE re-implementation. DR-LightGCN is in the Holm family only on Coat and KuaiRec; on Yahoo! R3 and KuaiRand it was not tested for the final method. Every paired difference against a graph baseline is significant ($p_{\text{Holm}} \leq 0.0185$) except one: LightGCN-pt on KuaiRec, +1.31 [3.34], $p_{\text{Holm}} = 0.4409$. Apart from LightGCN-pt, the largest adjusted p-value is 0.0185 (KuaiRand, r-AdjNorm).
2. **Yahoo! R3.** The final method reaches 0.6866 against 0.6742 for r-AdjNorm, the strongest baseline, a paired difference of +12.67 [6.01]

($\times 10^{-3}$, $p_{\text{Holm}} < 0.001$). It is the best method we ran on this dataset. The training-free DR adjacency and DRUP rows (0.6567) and the DR-JL and MRDR rows (about 0.52) are below it; these rows were not tested in the Holm family.

3. **KuaiRand.** The final method reaches 0.4613 against 0.4590 for r-AdjNorm, a paired difference of +1.94 [1.61] ($p_{\text{Holm}} = 0.0185$). The effect is small. Validation chose $\beta = 0$ in four of five splits, so in the selected runs the gain comes mainly from the weighting. The $\beta = 0$ job alone reaches 0.4623 on the test set, above the validated combination, so the validation choice is not always the test-best choice.
4. **Coat.** The final method reaches 0.6015 against 0.5812 for iALS, +20.02 [19.74] ($p_{\text{Holm}} = 0.0469$) in the main run. This difference does not survive retraining (Section 6.2), and the difference from iALS is not significant across seeds.
5. **KuaiRec.** The final method reaches 0.6285. It is below DR-JL (0.6332; −4.56 [2.24], $p_{\text{Holm}} < 0.001$) and MRDR (0.6328; −4.34 [2.16], $p_{\text{Holm}} < 0.001$), and below the training-free DR adjacency and DRUP operators (0.6378, not tested). Apart from DR-JL and MRDR it has the highest mean in the family, but the difference to LightGCN-pt is not significant. Section 6.5 discusses the training-free operators.
6. **Scale.** Against the strongest graph baseline by mean, the paired differences are +32.68 (Coat, DR-LightGCN), +12.67 (Yahoo! R3, r-AdjNorm), +1.94 (KuaiRand, r-AdjNorm) and +1.31 (KuaiRec, LightGCN-pt, not significant), in units of 10^{-3} nDCG. The paired differences therefore range from 1.3 to 32.7 thousandths of nDCG.

6.2. Robustness to the training seed

Table 5 retrains the selected configuration of every split with seeds 1, 2 and 3 and re-selects only the epoch on validation data. Seed 0 is the main run. Each dataset has a Holm family of two comparisons against its two strongest baselines in the main run, chosen by main-run test mean (Section 5.5).

On Coat the seed-averaged final method reaches 0.5911 ± 0.0049 against 0.5862 ± 0.0029 for iALS. The paired difference is +3.39 [20.70] with $p_{\text{Holm}} = 0.747$, which is not significant. The Coat claim is therefore that the difference from iALS is not significant across seeds. Against DR-LightGCN the difference remains large and significant (+33.36 [17.31], $p_{\text{Holm}} = 0.0004$). The spread of the final method across seeds on Coat (sd 0.0049, ddof = 0) is larger

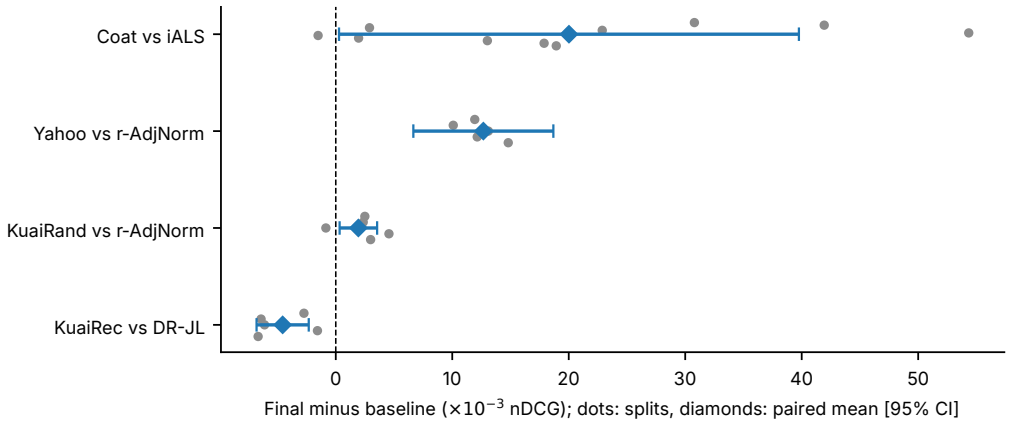


Figure 1: Per-split test difference between the final method and the strongest baseline of each dataset (Coat: iALS; Yahoo! R3 and KuaiRand: r-AdjNorm; KuaiRec: DR-JL). Dots are the splits (five, or ten on Coat); diamonds are the paired per-user mean difference from Table 3 with its 95% CI half-width. Units are $\times 10^{-3}$ nDCG.

than that of iALS (0.0029), and the seed-0 value of 0.6015 is the highest of the four seeds. The main-run advantage over iALS on Coat should therefore be read as a single-run result.

On Yahoo! R3 the seed-averaged final method reaches 0.6881 ± 0.0005, against 0.6736 for r-AdjNorm (+14.76 [5.23], $p_{\text{Holm}} < 0.001$) and 0.6691 for LightGCN (+20.02 [5.08], $p_{\text{Holm}} < 0.001$). On KuaiRand it reaches 0.4604 ± 0.0003, against 0.4580 for r-AdjNorm (+2.31 [1.23], $p_{\text{Holm}} = 0.0002$) and 0.4555 for LightGCN (+4.87 [1.12], $p_{\text{Holm}} < 0.001$). Both gains are small in absolute terms.

On KuaiRec the seed-averaged final method reaches 0.6290 ± 0.0002, against 0.6298 for DR-JL and 0.6304 for MRDR. The differences are -1.03 [1.64] ($p_{\text{Holm}} = 0.2213$) and -1.56 [1.47] ($p_{\text{Holm}} = 0.0760$). The significant deficit of the main run therefore becomes a gap of 1.0 to 1.6 thousandths of nDCG that is not significant over the retraining seeds, although the seed-averaged mean is lower than both. Both statements are reported: below DR-JL and MRDR in the main run, and not significantly different from them across retraining seeds.

6.3. Where does the gain come from?

Table 6 and Figure 2 separate the two ingredients: the weights on the sampled negatives and the fixed offset. Each row is one stored job. The rows are not matched in their configuration grids, and the ablation rows carry no significance tests. The one test in this section is the KuaiRec decomposition test below.

Weights.. On Coat the full weighting (Q-LightGCN, no offset) reaches 0.5912, against 0.5683 for LightGCN. Replacing Q by the imputation alone, $\hat{Y}$, gives 0.5937; replacing it by rescaled item popularity gives 0.5751; and a random permutation of the Q values, which keeps their marginal distribution, gives 0.5689, the LightGCN level. On Yahoo! R3 the weighted model reaches 0.6829 (learning rate 3×10^{-3} only), against 0.6708 for LightGCN, popularity-only weights give 0.6550, and the shuffled control gives 0.6689. The shuffled control removes the gain on both datasets, so the gain is not a rescaling effect of the weights. Popularity-only weights are weaker than Q on both datasets, and on Yahoo! R3 they are below LightGCN. On Coat, dropping the propensity from Q changes little (0.5937 against 0.5912), so in this comparison the information appears to lie in the imputation. This is an observation only: the grids are not matched and no test was run. The Coat weighted runs use 108 configurations (with fn $\in \{0.5, 1\}$) against 54 for the ablations, so these two numbers are not grid-matched.

Offset.. All offset ablations are on Coat. LightGCN with the offset only (P-LightGCN, $\beta \in \{0.5, 1\}$) reaches 0.6020, with the weights as well 0.6039 ($\beta \in \{0.5, 1\}$) and 0.6046 ($\beta \in \{1, 2\}$), and the validated combination 0.6015. On Coat, with unmatched grids and no test, the offset alone recovers essentially all of the gain, and the weighting adds little on top of it (0.6020 against 0.6039). Whether this holds on the other datasets is untested: offset-only runs exist only for Coat.

Imputation model.. On Coat a rank-16 low-rank imputation for Q and the offset gives 0.5887 without offset and 0.5897 with it, against 0.5912 and 0.6039 for the additive imputation. The more flexible model is worse on this dataset. Q-iALS, iALS with the same weights, reaches 0.5871 against 0.5812 for iALS, using 48 and 24 configurations. It was not run on the larger datasets, where the dense solve is too slow. We report the Q-iALS result as an observation and make no significance claim.

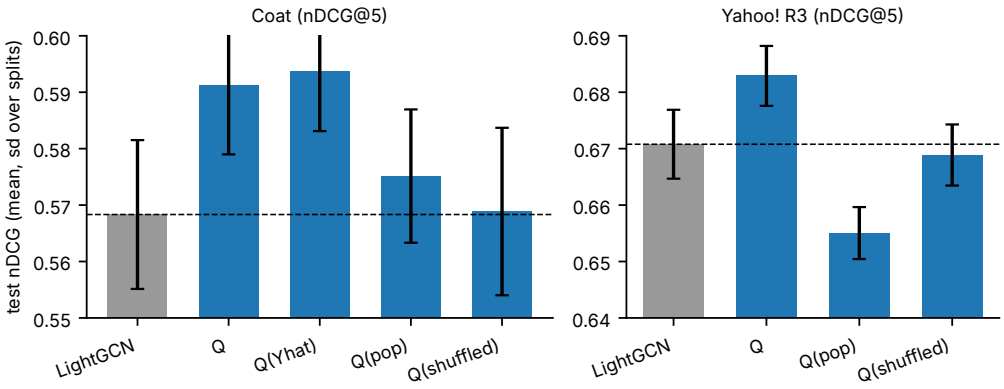


Figure 2: Ablations of the weights on Coat (left) and Yahoo! R3 (right): test nDCG@5, mean over splits, with standard deviation over splits. The dashed line is LightGCN. The Coat weighted model uses 108 configurations and the Yahoo! R3 weighted model a single learning rate (see Table 6). The Yahoo! R3 imputation-only variant was not run.

Decomposition across datasets.. Table 7 compares the weighting-only selection with the validated combination on every dataset. On Yahoo! R3 the weighting-only selection reaches 0.6837 and the combination 0.6866; on KuaiRand 0.4623 (the $\beta = 0$ job) against 0.4613; on KuaiRec 0.6095 against 0.6285. Read this way, the weighting carries most of the Yahoo! R3 and KuaiRand results. On KuaiRec the validated combination exceeds the weighting-only selection by 0.019. This is a decomposition of selected runs rather than a matched ablation. On KuaiRec the weighting alone is not significantly better than LightGCN (+1.39 [1.38], $p_{\text{Holm}} = 0.0967$) and has a lower mean than NAVIP, DR-LightGCN and iALS.

6.4. Comparison with AdvInfoNCE

Table 8 compares the final method with our re-implementation of AdvInfoNCE. The implementation is simplified: the adversary budget is implicit (the number of adversarial rounds), there is no KL-ball projection, and the grids differ across datasets. Our numbers are for this re-implementation and are not those of the original paper. The final method is higher on every dataset: 0.6015 against 0.5222 on Coat (+78.50 [27.63]), 0.6866 against 0.6477 on Yahoo! R3 (+37.85 [8.48]), 0.4613 against 0.4480 on KuaiRand (+12.79 [2.50]) and 0.6285 against 0.5776 on KuaiRec (+49.32 [6.57]), with

$p_{\text{Holm}} < 0.001$ in all four cases. The grids favour the re-implementation on Coat, which has 216 configurations, and disfavour it on the other datasets, which have 8 or 16. In our hands the contrastive family is weak on Coat: plain InfoNCE-LightGCN reaches 0.5290 there, and the DR-weighted variant 0.5531.

6.5. The walk correction and DRUP

Table 9 and Figure 3 report the walk correction of DRUP, which removes repeated-edge terms from multi-hop doubly robust propagation, and the validated ensemble with DRUP.

On real data the correction does not help accuracy at matched configurations. On Coat, DRUP minus DR adjacency is negative for every value of β in the grid, from -0.33×10^{-3} at $\beta = 0.03$ to -8.17×10^{-3} at $\beta = 1000$, and is negative in all ten splits for $\beta \geq 0.3$. On Yahoo! R3 the differences are small and negative (-0.02 to -0.32×10^{-3}).

In the semi-synthetic KuaiRec study, with true propensities and fixed degree weights, DRUP minus DR nDCG@20 is positive at $\beta \in \{0.3, 1, 3\}$ in all six cells. At $\beta = 0.3$ the differences range from $+12.5$ to $+19.6 \times 10^{-3}$. The sign reverses at large β in the two cells with $\gamma = 1.0$ and $\kappa \in \{0.5, 1.5\}$, reaching -5.35×10^{-3} at $\beta = 1000$ (Supplement S4 gives the design). With imputation-based degree estimates, in the one stored cell with $\kappa = 1.5$ and $\gamma = 1.0$, DRUP minus DR is $+2.07$, $+6.25$ and $+2.90 \times 10^{-3}$ at $\beta = 0.3$, 1 and 3, and -5.20 and -9.36×10^{-3} at $\beta = 10$ and 1000.

The training-free operators settle the KuaiRec comparison. DRUP reaches 0.6378 and the DR adjacency on which it is built also reaches 0.6378; a five-hop DR adjacency reaches 0.6379. The credit for the strongest KuaiRec result therefore goes to doubly robust propagation, not to the walk correction. On Coat, DRUP is weak (0.5467). On Yahoo! R3 and KuaiRand the DRUP row equals the IPS adjacency (0.6567 and 0.4492), so the doubly robust half of DRUP is tested only on Coat and KuaiRec.

The validated ensemble $s = (1 - w)\, z(A) + w\, z(\text{DRUP})$ does not add to either component. On Coat w is 0 in nine splits and 0.3 in one, and the ensemble reaches 0.6031, +1.79 [2.39] over the final method ($p_{\text{Holm}} = 0.1421$) and +21.81 [20.25] over iALS ($p_{\text{Holm}} = 0.0823$). On KuaiRec w is 1.0 in four splits and 0.8 in one, and the ensemble reaches 0.6376, which is -0.18 [0.19] against DRUP ($p_{\text{Holm}} = 0.0664$). Validation picks one component in each dataset; the two are not complementary here.

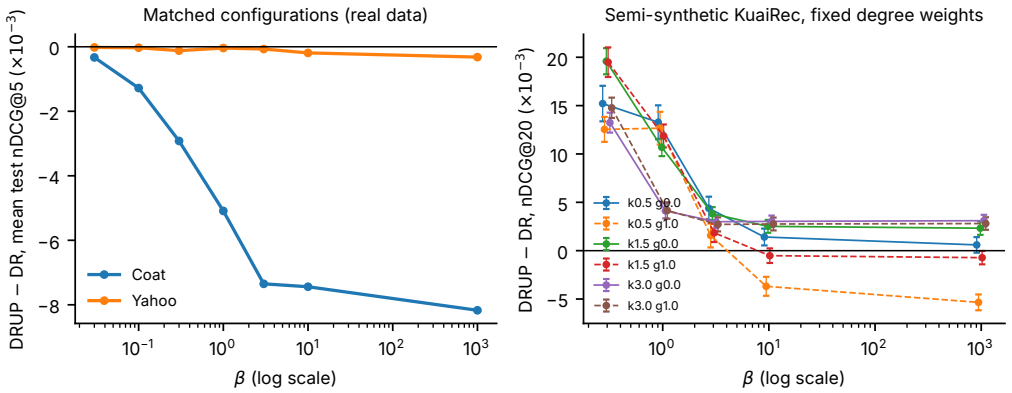


Figure 3: Left: matched-configuration DRUP minus DR adjacency on Coat and Yahoo! R3 against β (log scale), from Table 9. Right: semi-synthetic DRUP minus DR nDCG@20 on KuaiRec for the six cells of the design, with 95% CI half-widths; dashed lines are cells with $\gamma = 1.0$. Zero lines are shown in both panels.

7. Discussion

7.1. Why the method works

The posterior Q depends on the covariates only, through $\hat{p}$ and $\hat{Y}$; it does not depend on the realised exposure. Where the estimated exposure propensity is small, Q is close to the imputation $\hat{Y}$, so sampled negatives are down-weighted in proportion to the estimated probability that the user would like the item. Where the estimated propensity is close to one, $Q = \hat{Y}(1 - \hat{p})/(1 - \hat{p}\hat{Y})$ is close to zero only if $\hat{Y}$ is also small. We did not test whether pairs with high estimated exposure propensity are in fact true negatives. Rarely exposed items are the ones most likely to be false negatives under exposure bias, and this is the motivation for the weighting. ExpoMF (Liang et al., 2016) computes a related posterior for unclicked items inside matrix factorisation. Here the posterior has a closed form in the nuisances of doubly robust estimation and enters a pairwise graph loss.

The offset contributes a different signal. It adds a population-level preference estimate, the user and item effects of an IPS-weighted additive model fitted once on the log. We conjecture that a BPR objective on the observed positives has difficulty learning such an effect from a biased log, because the exposed items are the ones it sees. This conjecture was not tested directly.

The shuffled-Q control is a check against the explanation that the gain comes from the variance of the weights alone: a random permutation of Q has the same marginal distribution and removes the gain. On Coat the offset alone reaches essentially the same accuracy as the combination, so, with unmatched grids and no test, the offset appears to be the larger ingredient there, while on Yahoo! R3 and KuaiRand the weighting carries most of the gain in the selected runs.

7.2. When it does not help

KuaiRec is the setting in which the method is weakest. Its log is dense (13.4% of the matrix is observed) and the test matrix is fully observed, with 3,314 candidates per user. These are the features that differ most from the other three datasets; we have no direct test of which of them causes the gap. In this setting the pointwise doubly robust methods (DR-JL and MRDR) and training-free DR propagation are stronger. Pointwise training does not close the gap either: LightGCN-pn reaches 0.5992–0.5995 and DR-LightGCN-pn 0.5992–0.6027 across the two learning rates, the latter being the pointwise loss with the same Q weights on negatives. Both are far below DR-JL and MRDR. On Coat the difference from iALS is not significant across seeds, and iALS has a higher main-run mean than every graph baseline there.

7.3. Practical guidance

The following guidance summarises the selections observed in these experiments. It is based on the observed selections and was not tested as a procedure. For a practitioner with a biased log and a uniform validation sample, four steps follow from the results. First, fit the additive IPS imputation and the exposure model on the log, and check that Q is informative with a shuffled control; if the shuffled model matches the plain model, the weights carry no information in this comparison, and an offset-only model is the natural alternative to test. Second, select the offset weight on validation data and include $\beta = 0$ in the grid, because the validation choice is not always the test-best one (KuaiRand). Third, report the comparison against the strongest simple baselines, including iALS, because they remain close on some datasets. Fourth, on dense logs with a fully observed target, compare against training-free DR propagation, which needs no gradient training; on KuaiRec it reaches 0.6378, against 0.6285 for the trained method.

7.4. Limitations

1. **Unequal tuning budgets.** The final method is the union of several jobs. It was tuned with 324 configurations on Coat, against 24 for iALS, 54 for LightGCN and 108 for r-AdjNorm; with 84 on Yahoo! R3 against 72 for r-AdjNorm; with 30 on KuaiRand against 12 for r-AdjNorm; and with 120 on KuaiRec against 6 for DR-JL and MRDR. Selecting over more configurations can favour the method. The AdvInfoNCE grid is largest on Coat (216 configurations) and smallest elsewhere (8 or 16), so the comparison with it is unequal in both directions.
2. **AdvInfoNCE is a simplified re-implementation.** The adversary budget is implicit (the number of adversarial rounds, with $\eta \in \{4, 12\}$), there is no KL-ball projection, and only an embedding-based adversary was tried. Its numbers are not those of the original paper.
3. **Small effects.** The KuaiRand gain is $+1.94\times10^{-3}$ in the main run and $+2.31\times10^{-3}$ over the seeds. The paired differences against the strongest graph baseline range from 0.0013 to 0.0327 nDCG (Section 6.1), and the confidence-interval half-widths on Coat are between 0.017 and 0.028.
4. **Main run versus seed runs.** Table 3 is a single training seed with full selection. The seed runs retrain only the selected configurations and compare against two baselines per dataset, chosen by main-run test means. Over these runs the Coat difference from iALS and the KuaiRec differences from DR-JL and MRDR are not significant.
5. **KuaiRec is lost to the doubly robust baselines.** In the main run the method is below DR-JL and MRDR, and in all runs it is below training-free DRUP and DR adjacency (0.6378).
6. **One imputation family.** The propensity and the imputation come from one additive model, and on Coat from one low-rank alternative. Other imputation families are untested.
7. **Ablations are mostly on Coat.** The offset-only, imputation-only, low-rank and Q-iALS ablations were run on Coat only. Yahoo! R3 has the popularity and shuffled controls only, and KuaiRand and KuaiRec have only the decomposition of the selected runs. No significance tests were run for the ablations, except the single KuaiRec decomposition test reported in Section 6.3.
8. **Coat propensities may leak.** The Coat propensities were estimated by the dataset authors with the uniform sample (Schnabel et al., 2016) that we also use for validation and testing. This is a possible leak that

affects Coat only. The same propensities are used by every propensity-based method on Coat, so the Coat comparison shares this input.

9. **Exposure semantics.** On Coat and Yahoo! R3, "exposure" is self-selection by users rather than platform exposure. The derivation of Q assumes that exposure is independent of the outcome given the covariates of the propensity model.
10. **Overlapping splits.** Per-user t-tests over overlapping splits treat the log and the fitted nuisances as fixed. They can be anti-conservative (Nadeau and Bengio, 2003).
11. **Scale.** Q and the offset are dense $m \times n$ matrices. The implementation therefore fits the catalogues used here, the largest being KuaiRec with 7,176 users and 10,728 items. Computing Q on the fly from the nuisances would remove this limit; it is not implemented.
12. **Methods not compared.** StableDR, DICE, CausE, AutoDebias and large language model based recommenders are not in the comparison. Q-iALS was not run on the larger datasets.
13. **Validation choice is not always test-best.** On KuaiRand the $\beta = 0$ job reaches 0.4623 on test, against 0.4613 for the validated combination.

8. Conclusion

We proposed a weighting and an offset for graph collaborative filtering under exposure bias. Sampled BPR negatives are down-weighted by one minus the probability that an unlogged pair is positive, and the standardised logit of an inverse-propensity-weighted additive imputation enters the score as a fixed offset. Both nuisances are fitted once on the biased log. Uniformly collected data are used only for validation and testing, except that the shipped Coat propensities were estimated by the dataset authors from a uniform sample.

On Coat, Yahoo! R3, KuaiRand and KuaiRec the final method (QP-LightGCN) has a higher mean than every graph baseline we ran, and the differences are significant except against LightGCN-pt on KuaiRec. It is the best method we ran on Yahoo! R3 and KuaiRand. In the main run it is above iALS on Coat and below DR-JL and MRDR on KuaiRec. Across retraining seeds these differences are not significant. Training-free doubly robust propagation remains the best method we ran (0.6378). The ablations show that the gain depends on an informative posterior: a shuffled control

removes it on Coat and Yahoo! R3. On Coat, with unmatched grids, the offset alone recovers most of the improvement.

Four extensions follow from the limitations. Computing the posterior on the fly would remove the memory limit of dense matrices on large catalogues. Matched offset-only runs on all four datasets would test the offset claim beyond Coat. A faithful reproduction of AdvInfoNCE, with its original adversary, would replace our simplified re-implementation. Propensities estimated without the uniform data used for testing would remove the possible leak on Coat.

References

Anelli, V.W., Malitesta, D., Pomo, C., Bellogin, A., Di Sciascio, E., Di Noia, T., 2023. Challenging the myth of graph collaborative filtering: a reasoned and reproducibility-driven analysis, in: Proceedings of the 17th ACM Conference on Recommender Systems, ACM. pp. 350–361. doi:10.1145/3604915.3609489.

Bai, T., Chen, W., Yang, C., Shi, C., 2025. Invariant debiasing learning for recommendation via biased imputation. Information Processing & Management 62, 104028. doi:10.1016/j.ipm.2024.104028.

Bonner, S., Vasile, F., 2018. Causal embeddings for recommendation, in: Proceedings of the 12th ACM Conference on Recommender Systems, pp. 104–112. doi:10.1145/3240323.3240360.

Boratto, L., Fenu, G., Marras, M., 2021. Connecting user and item perspectives in popularity debiasing for collaborative recommendation. Information Processing & Management 58, 102387. doi:10.1016/j.ipm.2020.102387.

Chen, J., Dong, H., Qiu, Y., He, X., Xin, X., Chen, L., Lin, G., Yang, K., 2021. AutoDebias: Learning to debias for recommendation, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 21–30. doi:10.1145/3404835.3462919.

Chen, J., Dong, H., Wang, X., Feng, F., Wang, M., He, X., 2023. Bias and debias in recommender system: A survey and future directions. ACM Transactions on Information Systems 41, 1–39. doi:10.1145/3564284.

Chen, J., Wu, J., Chen, J., Xin, X., Li, Y., He, X., 2024. How graph convolutions amplify popularity bias for recommendation? Frontiers of Computer Science 18. doi:10.1007/s11704-023-2655-2.

Choi, J., Hong, S., Park, N., Cho, S.B., 2023. Blurring-sharpening process models for collaborative filtering, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 1096–1106. doi:10.1145/3539618.3591645.

Chuang, C.Y., Robinson, J., Lin, Y.C., Torralba, A., Jegelka, S., 2020. Debiased contrastive learning, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

Dai, Q., Li, H., Wu, P., Dong, Z., Zhou, X.H., Zhang, R., Zhang, R., Sun, J., 2022. A generalized doubly robust learning framework for debiasing post-click conversion rate prediction, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM. pp. 252–262. doi:10.1145/3534678.3539270.

Ding, J., Quan, Y., Yao, Q., Li, Y., Jin, D., 2020. Simplify and robustify negative sampling for implicit collaborative filtering, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

Dudík, M., Langford, J., Li, L., 2011. Doubly robust policy evaluation and learning, in: Proceedings of the 28th International Conference on Machine Learning, pp. 1097–1104.

Ferrari Dacrema, M., Cremonesi, P., Jannach, D., 2019. Are we really making much progress? A worrying analysis of recent neural recommendation approaches, in: Proceedings of the 13th ACM Conference on Recommender Systems, ACM. pp. 101–109. doi:10.1145/3298689.3347058.

Gao, C., Li, S., Lei, W., Chen, J., Li, B., Jiang, P., He, X., Mao, J., Chua, T.S., 2022a. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems [data set], in: Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM), pp. 540–550. doi:10.1145/3511808.3557220.

Gao, C., Li, S., Zhang, Y., Chen, J., Li, B., Lei, W., Jiang, P., He, X., 2022b. KuaiRand: An unbiased sequential recommendation dataset with

randomly exposed videos [data set], in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM), pp. 3953–3957. doi:10.1145/3511808.3557624.

Gao, C., Zheng, Y., Wang, W., Feng, F., He, X., Li, Y., 2024. Causal inference in recommender systems: A survey and future directions. ACM Transactions on Information Systems 42, 1–32. doi:10.1145/3639048.

Guo, S., Zou, L., Liu, Y., Ye, W., Cheng, S., Wang, S., Chen, H., Yin, D., Chang, Y., 2021. Enhanced doubly robust learning for debiasing post-click conversion rate estimation, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 275–284. doi:10.1145/3404835.3462917.

He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., Wang, M., 2020. LightGCN: Simplifying and powering graph convolution network for recommendation, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 639–648.

Holm, S., 1979. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6, 65–70.

Hu, Y., Koren, Y., Volinsky, C., 2008. Collaborative filtering for implicit feedback datasets, in: 2008 Eighth IEEE International Conference on Data Mining (ICDM), pp. 263–272. doi:10.1109/ICDM.2008.22.

Islam, M.A., Faruk, A.S., Medya, S., Zheleva, E., 2026. Debiasing message passing to mitigate popularity bias in GNN-based collaborative filtering. arXiv:2605.11145. preprint.

Kim, M., Oh, J., Do, J., Lee, S., 2022. Debiasing neighbor aggregation for graph neural network in recommender systems, in: Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM), pp. 4128–4132. doi:10.1145/3511808.3557576.

Krichene, W., Rendle, S., 2020. On sampled metrics for item recommendation, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1748–1757.

Li, H., Zheng, C., Wu, P., 2023. StableDR: Stabilized doubly robust learning for recommendation on data missing not at random, in: International Conference on Learning Representations (ICLR).

Liang, D., Charlin, L., McInerney, J., Blei, D.M., 2016. Modeling user exposure in recommendation, in: Proceedings of the 25th International Conference on World Wide Web, pp. 951–961. doi:10.1145/2872427.2883090.

Mao, K., Zhu, J., Xiao, X., Lu, B., Wang, Z., He, X., 2021. UltraGCN: Ultra simplification of graph convolutional networks for recommendation, in: Proceedings of the 30th ACM International Conference on Information and Knowledge Management, pp. 1253–1262. doi:10.1145/3459637.3482291.

Marlin, B.M., Zemel, R.S., 2009. Collaborative prediction and ranking with non-random missing data [data set], in: Proceedings of the Third ACM Conference on Recommender Systems (RecSys), pp. 5–12. doi:10.1145/1639714.1639717.

Menon, A.K., Jayasumana, S., Rawat, A.S., Jain, H., Veit, A., Kumar, S., 2021. Long-tail learning via logit adjustment, in: International Conference on Learning Representations (ICLR).

Nadeau, C., Bengio, Y., 2003. Inference for the generalization error. Machine Learning 52, 239–281. doi:10.1023/a:1024068626366.

Okamura, H., Maeda, K., Togo, R., Ogawa, T., Haseyama, M., 2024. Flexibly manipulating popularity bias for tackling trade-offs in recommendation. Information Processing & Management 61, 103606. doi:10.1016/j.ipm.2023.103606.

Que, Y., Zhang, Y., Zhao, X., Ma, C., 2025. Causality-aware graph aggregation weight estimator for popularity debiasing in top-k recommendation, in: Proceedings of the 34th ACM International Conference on Information and Knowledge Management, ACM. pp. 2471–2481. doi:10.1145/3746252.3761155.

Rendle, S., Freudenthaler, C., Gantner, Z., Schmidt-Thieme, L., 2009. BPR: Bayesian personalized ranking from implicit feedback, in: Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI), pp. 452–461.

Rendle, S., Zhang, L., Koren, Y., 2019. On the difficulty of evaluating baselines: A study on recommender systems. arXiv:1905.01395. preprint.

Saito, Y., 2020. Unbiased pairwise learning from biased implicit feedback, in: Proceedings of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR), pp. 5–12.

Saito, Y., Yaginuma, S., Nishino, Y., Sakata, H., Nakata, K., 2020. Unbiased recommender learning from missing-not-at-random implicit feedback, in: Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM), pp. 501–509.

Schnabel, T., Swaminathan, A., Singh, A., Chandak, N., Joachims, T., 2016. Recommendations as treatments: Debiasing learning and evaluation [data set], in: Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 1670–1679.

Shen, Y., Wu, Y., Zhang, Y., Shan, C., Zhang, J., Letaief, K.B., Li, D., 2021. How powerful is graph convolution for recommendation?, in: Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM), pp. 1619–1629.

Steck, H., 2010. Training and testing of recommender systems on data missing not at random, in: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 713–722.

Steck, H., 2019. Embarrassingly shallow autoencoders for sparse data, in: The World Wide Web Conference, ACM. pp. 3251–3257. doi:10.1145/3308558.3313710.

Swaminathan, A., Joachims, T., 2015. The self-normalized estimator for counterfactual learning, in: Advances in Neural Information Processing Systems, pp. 3231–3239.

Wang, X., He, X., Wang, M., Feng, F., Chua, T.S., 2019a. Neural graph collaborative filtering, in: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 165–174. doi:10.1145/3331184.3331267.

Wang, X., Zhang, R., Sun, Y., Qi, J., 2019b. Doubly robust joint learning for recommendation on data missing not at random, in: Proceedings of the

36th International Conference on Machine Learning (ICML), pp. 6638–6647.

Wang, Z., Chen, X., Wen, R., Huang, S.L., Kuruoglu, E.E., Zheng, Y., 2020. Information theoretic counterfactual learning from missing-not-at-random feedback, in: Advances in Neural Information Processing Systems.

Wei, T., Feng, F., Chen, J., Wu, Z., Yi, J., He, X., 2021. Model-agnostic counterfactual reasoning for eliminating popularity bias in recommender system, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 1791–1800. doi:10.1145/3447548.3467289.

Wu, J., Wang, X., Feng, F., He, X., Chen, L., Lian, J., Xie, X., 2021. Self-supervised graph learning for recommendation, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 726–735. doi:10.1145/3404835.3462862.

Wu, S., Sun, F., Zhang, W., Xie, X., Cui, B., 2023. Graph neural networks in recommender systems: A survey. ACM Computing Surveys 55, 1–37. doi:10.1145/3535101.

Yu, J., Yin, H., Xia, X., Chen, T., Cui, L., Nguyen, Q.V.H., 2022. Are graph augmentations necessary?: Simple graph contrastive learning for recommendation, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 1294–1303. doi:10.1145/3477495.3531937.

Zhang, A., Ma, W., Wang, X., Chua, T.S., 2022. Incorporating bias-aware margins into contrastive loss for collaborative filtering, in: Advances in Neural Information Processing Systems 35 (NeurIPS 2022).

Zhang, A., Sheng, L., Cai, Z., Wang, X., Chua, T.S., 2023. Empowering collaborative filtering with principled adversarial contrastive loss, in: Advances in Neural Information Processing Systems 36 (NeurIPS 2023).

Zhang, Y., Feng, F., He, X., Wei, T., Song, C., Ling, G., Zhang, Y., 2021. Causal intervention for leveraging popularity bias in recommendation, in: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 11–20. doi:10.1145/3404835.3462875.

Zhao, M., Wu, L., Liang, Y., Chen, L., Zhang, J., Deng, Q., Wang, K., Shen, X., Lv, T., Wu, R., 2022. Investigating accuracy-novelty performance for graph-based collaborative filtering, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 50–59. doi:10.1145/3477495.3532005.

Zheng, Y., Gao, C., Li, X., He, X., Li, Y., Jin, D., 2021. Disentangling user interest and conformity for recommendation with causal embedding, in: Proceedings of the Web Conference (WWW), pp. 2980–2991.

Zhou, H., Chen, H., Dong, J., Zha, D., Zhou, C., Huang, X., 2023. Adaptive popularity debiasing aggregator for graph collaborative filtering, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 7–17. doi:10.1145/3539618.3591635.

Table 3: Main comparison (seed 0, full selection). Test nDCG is mean ± sd over splits. Diff is the paired per-user difference of the final method minus the baseline, $\times 10^{-3}$, with the 95% CI half-width. p_{Holm} is adjusted within each dataset over the block's family (sizes 8, 6, 7, 9). Bold: best of the block; underline: second (KuaiRec: within the family). The final method is tuned over 324, 84, 30 and 120 configurations (Coat, Yahoo! R3, KuaiRand, KuaiRec), against 6 to 216 for the baselines (Configs column). Rows marked re-impl. use a simplified AdvInfoNCE (Section 5.3). On KuaiRec the final method ranks third within its Holm family.

Method	Test nDCG	Configs	Diff $\times 10^{-3}$ [95% CI]	p_{Holm}
Coat (nDCG@5; 10 splits)				
QP-LightGCN(final)	**0.6015 ± 0.0162**	324	–	–
iALS	0.5812 ± 0.0101	24	+20.02 [19.74]	0.0469
DR-LightGCN	0.5724 ± 0.0205	54	+32.68 [23.12]	0.0116
r-AdjNorm	0.5722 ± 0.0134	108	+28.27 [18.42]	0.0083
LightGCN	0.5683 ± 0.0132	54	+32.03 [17.27]	0.0016
NAVIP	0.5664 ± 0.0163	54	+33.62 [19.63]	0.0035
SimGCL	0.5528 ± 0.0141	108	+49.52 [23.99]	0.0004
MACR	0.5418 ± 0.0134	36	+61.46 [27.66]	< 0.001
AdvInfoNCE (re-impl.)	0.5222 ± 0.0204	216	+78.50 [27.63]	< 0.001
Yahoo! R3 (nDCG@5; 5 splits)				
QP-LightGCN(final)	**0.6866 ± 0.0034**	84	–	–
r-AdjNorm	0.6742 ± 0.0047	72	+12.67 [6.01]	< 0.001
LightGCN	0.6708 ± 0.0061	36	+16.77 [5.82]	< 0.001
NAVIP	0.6701 ± 0.0039	36	+17.51 [6.02]	< 0.001
iALS	0.6625 ± 0.0043	12	+24.88 [8.77]	< 0.001
AdvInfoNCE (re-impl.)	0.6477 ± 0.0048	16	+37.85 [8.48]	< 0.001
SimGCL	0.6348 ± 0.0048	12	+53.26 [10.05]	< 0.001
KuaiRand (nDCG@10; 5 splits)				
QP-LightGCN(final)	**0.4613 ± 0.0024**	30	–	–
r-AdjNorm	0.4590 ± 0.0024	12	+1.94 [1.61]	0.0185
LightGCN	0.4569 ± 0.0010	6	+3.93 [1.53]	< 0.001
NAVIP	0.4565 ± 0.0017	6	+4.36 [1.46]	< 0.001
iALS	0.4528 ± 0.0017	12	+8.98 [3.23]	< 0.001
AdvInfoNCE (re-impl.)	0.4480 ± 0.0017	8	+12.79 [2.50]	< 0.001
BPR-MF	0.4475 ± 0.0027	6	+13.99 [2.62]	< 0.001
SimGCL	0.4196 ± 0.0022	12	+42.20 [4.04]	< 0.001
KuaiRec (nDCG@20; 5 splits)				
DR-JL	**0.6332 ± 0.0026**	6	−4.56 [2.24]	< 0.001
MRDR	0.6328 ± 0.0020	6	−4.34 [2.16]	< 0.001
QP-LightGCN(final)	0.6285 ± 0.0022	120	–	–
LightGCN-pt	0.6271 ± 0.0023	36	+1.31 [3.34]	0.4409
DR-LightGCN	0.6212 ± 0.0071	6	+7.73 [2.01]	< 0.001
NAVIP	0.6186 ± 0.0024	6	+10.49 [2.28]	< 0.001
iALS	0.6133 ± 0.0027	12	+15.92 [3.03]	< 0.001
LightGCN	0.6085 ± 0.0028	36	+20.90 [2.88]	< 0.001
r-AdjNorm	0.6083 ± 0.0031	12	+20.68 [2.75]	< 0.001
AdvInfoNCE (re-impl.)	0.5776 ± 0.0036	8	+49.32 [6.57]	< 0.001

Beta per split: Coat 1.0 (5 splits), 2.0 (2), 0.5 (1), 0.0 (2); Yahoo! R3 1.0 (4), 0.0 (1); KuaiRand 0.0 (4), 0.5 (1); KuaiRec 1.0 (4), 2.0 (1). Training-free and other rows: Table 4.

Table 4: Training-free and other methods outside the Holm families of Table 3. Their differences to the final method were not tested. Test nDCG is mean ± sd over splits. DR adjacency is the doubly robust operator without the walk correction; DRUP adds the walk correction. On Yahoo! R3 and KuaiRand, DRUP, DR adjacency and IPS adjacency have identical means.

Dataset	Method	Test nDCG
Coat (nDCG@5)	DRUP	0.5467 ± 0.0235
	DR adjacency	0.5459 ± 0.0244
	MRDR	0.4869 ± 0.0187
	DR-JL	0.4811 ± 0.0164
Yahoo! R3 (nDCG@5)	DRUP	0.6567 ± 0.0063
	DR adjacency	0.6567 ± 0.0063
	DR-LightGCN	0.5528 ± 0.0091
	MRDR	0.5248 ± 0.0059
	DR-JL	0.5225 ± 0.0043
KuaiRand (nDCG@10)	DRUP	0.4492 ± 0.0013
	DR adjacency	0.4492 ± 0.0013
	DR-JL	0.3759 ± 0.0021
KuaiRec (nDCG@20)	DRUP	**0.6378 ± 0.0019**
	DR adjacency	0.6378 ± 0.0020

Table 5: Robustness to the training seed. Seed 0 is the main run of Table 3; seeds 1–3 retrain the selected configuration of each split and re-select only the epoch on validation data. Test nDCG per seed is the mean over splits. Each dataset has a Holm family of two paired comparisons (the two strongest baselines of the main run). The two baselines of each dataset are the two highest-scoring baselines of the main run by test mean, and they are retrained with seeds 1–3 in the same way as the final method. Diff is the paired per-user mean difference (not the difference of the two table means), $\times 10^{-3}$, with the 95% CI half-width in brackets.

Method	Seed 0	Seed 1	Seed 2	Seed 3	Mean ± sd (1–3)	sd (ddof 1)
Coat (nDCG@5)						
QP-LightGCN(final)	0.6015	0.5980	0.5879	0.5874	0.5911 ± 0.0049	0.0060
iALS	0.5812	0.5900	0.5829	0.5858	0.5862 ± 0.0029	0.0036
DR-LightGCN	0.5724	0.5550	0.5632	0.5645	0.5609 ± 0.0042	0.0051
vs iALS: +3.39 [20.70], $p_{\text{Holm}} = 0.747$						
vs DR-LightGCN: +33.36 [17.31], $p_{\text{Holm}} = 0.0004$						
Yahoo! R3 (nDCG@5)						
QP-LightGCN(final)	0.6866	0.6874	0.6882	0.6886	0.6881 ± 0.0005	0.0006
r-AdjNorm	0.6742	0.6711	0.6752	0.6746	0.6736 ± 0.0018	0.0022
LightGCN	0.6708	0.6671	0.6700	0.6702	0.6691 ± 0.0014	0.0018
vs r-AdjNorm: +14.76 [5.23], $p_{\text{Holm}} < 0.001$						
vs LightGCN: +20.02 [5.08], $p_{\text{Holm}} < 0.001$						
KuaiRand (nDCG@10)						
QP-LightGCN(final)	0.4613	0.4602	0.4601	0.4608	0.4604 ± 0.0003	0.0004
r-AdjNorm	0.4590	0.4582	0.4576	0.4582	0.4580 ± 0.0003	0.0003
LightGCN	0.4569	0.4557	0.4547	0.4561	0.4555 ± 0.0006	0.0007
vs r-AdjNorm: +2.31 [1.23], $p_{\text{Holm}} = 0.0002$						
vs LightGCN: +4.87 [1.12], $p_{\text{Holm}} < 0.001$						
KuaiRec (nDCG@20)						
QP-LightGCN(final)	0.6285	0.6292	0.6287	0.6291	0.6290 ± 0.0002	0.0003
DR-JL	0.6332	0.6304	0.6308	0.6283	0.6298 ± 0.0011	0.0013
MRDR	0.6328	0.6305	0.6291	0.6315	0.6304 ± 0.0010	0.0012
vs DR-JL: −1.03 [1.64], $p_{\text{Holm}} = 0.2213$						
vs MRDR: −1.56 [1.47], $p_{\text{Holm}} = 0.0760$						

Table 6: Ablations of the weights and of the offset (Section 6.3). Test nDCG@5 is mean ± sd over splits (ddof = 0), with the number of configurations in brackets. Each row is one stored job. Dashes mark settings that were not run. No significance tests were run for these ablations. The decomposition of the selected runs on all datasets is in Table 7.

Variant	Coat	Yahoo! R3
Panel A: weights only ($\beta = 0$)		
LightGCN	0.5683 ± 0.0132 (54)	0.6708 ± 0.0061 (36)
Q-LightGCN (no offset)	0.5912 ± 0.0122 (108)[a]	0.6829 ± 0.0053 (12)[b]
Q($\hat{Y}$) (imputation only)	0.5937 ± 0.0106 (54)	–
Q(pop) (popularity only)	0.5751 ± 0.0118 (54)	0.6550 ± 0.0046 (12)
Q(shuffled), negative control	0.5689 ± 0.0148 (54)	0.6689 ± 0.0054 (12)
Panel B: offset and imputation (Coat only)		
P-LightGCN (offset only, $\beta \in \{0.5, 1\}$)	0.6020 ± 0.0148 (108)	–
QP-LightGCN, $\beta \in \{0.5, 1\}$	0.6039 ± 0.0139 (108)	–
QP-LightGCN, $\beta \in \{1, 2\}$	0.6046 ± 0.0177 (108)	–
QP-LightGCN, validated β (final)	0.6015 ± 0.0162 (324)	–
Q-LightGCN, low-rank $\hat{Y}$ (rank 16)	0.5887 ± 0.0132 (54)	–
QP-LightGCN, low-rank $\hat{Y}$ (rank 16)	0.5897 ± 0.0155 (108)	–
Q-iALS (iALS: 0.5812, 24)	0.5871 ± 0.0113 (48)	–

[a] 108 configurations with fn $\in \{0.5, 1\}$, against 54 with fn = 1 for the other Coat weight ablations, so this row is not grid-matched. [b] Learning rate 3×10^{-3} only (12 configurations); the Yahoo! R3 LightGCN row is taken over three learning rates (36 configurations), so it is not grid-matched either.

Table 7: Weighting only versus the validated combination on every dataset (Section 6.3). Test nDCG (nDCG@5, @5, @10 and @20 for the four datasets, respectively), mean over splits. These are decompositions of selected runs, not matched ablations, and no standard deviations are reported for the combined selections.

Variant	Coat	Yahoo! R3	KuaiRand	KuaiRec
Q-LightGCN, $\beta = 0$	0.5912	0.6837[a]	0.4623	0.6095[a]
QP-LightGCN, validated β	0.6015	0.6866	0.4613	0.6285

[a] Selected per split among the learning-rate jobs (three on Yahoo! R3, two on KuaiRec). On KuaiRand the $\beta = 0$ job alone gives 0.4623, above the validated combination (0.4613), because validation picks $\beta > 0$ in one split.

Table 8: Comparison with the AdvInfoNCE re-implementation (Section 6.4). This is not the original method: the adversary budget is implicit (the number of adversarial rounds), there is no KL-ball projection, and the grids differ across datasets (given below each dataset). Diff is the paired per-user difference of the final method minus AdvInfoNCE, $\times 10^{-3}$, with the 95% CI half-width; p_{Holm} is adjusted within each dataset.

Dataset	AdvInfoNCE	Configs	QP-LightGCN	Diff [CI]	p_{Holm}
Coat	0.5222 ± 0.0204	216	0.6015	+78.50 [27.63]	< 0.001
$\tau \in \{0.2, 0.75\}$, $\eta \in \{4, 12\}$, 3 learning rates, 3 weight decays dims $\{32, 64\}$, layers $\{1, 2, 3\}$					
Yahoo! R3	0.6477 ± 0.0048	16	0.6866	+37.85 [8.48]	< 0.001
$\tau \in \{0.1, 0.28\}$, $\eta \in \{4, 12\}$, lr {3e-3, 1e-3} wd $\{10^{-5}, 10^{-4}\}$, dim 128, 2 layers					
KuaiRand	0.4480 ± 0.0017	8	0.4613	+12.79 [2.50]	< 0.001
$\tau \in \{0.1, 0.28\}$, $\eta \in \{4, 12\}$, lr $\{10^{-2}, 3\text{e-}3\}$, wd 10^{-4}					
KuaiRec	0.5776 ± 0.0036	8	0.6285	+49.32 [6.57]	< 0.001
$\tau \in \{0.2, 2.0\}$, $\eta \in \{4, 12\}$, lr $\{10^{-2}, 3\text{e-}3\}$, wd 10^{-5}					
Context on Coat (not part of the comparison above)					
Plain InfoNCE	0.5290 ± 0.0199	108	–	–	–
DR-weighted InfoNCE	0.5531 ± 0.0094	216	–	–	–

Table 9: Walk correction and DRUP. Panel A: matched configurations, DRUP minus DR adjacency at the same configuration, for each value of β in the grid, on the real data. Mean differences are $\times 10^{-3}$ nDCG@5; "share < 0" is the fraction of the matched configurations with a negative mean test difference (1,440 on Coat, 512 on Yahoo! R3); "splits < 0" counts the splits in which the mean over configurations is negative. Panel B: validated score ensemble $s = (1-w)\, z(A) + w\, z(\text{DRUP})$, with z standardisation over unlogged items and w chosen per split on validation data; A is the final method. Holm family of 11 tests within each dataset.

	Coat			Yahoo! R3		
β	Diff	Share	Splits	Diff	Share	Splits
Panel A: matched configurations (DRUP minus DR)						
0.03	−0.33	0.49	7/10	−0.02	0.41	5/5
0.1	−1.28	0.52	9/10	−0.03	0.50	4/5
0.3	−2.92	0.54	10/10	−0.12	0.56	5/5
1.0	−5.09	0.74	10/10	−0.04	0.47	3/5
3.0	−7.35	0.74	10/10	−0.07	0.28	4/5
10.0	−7.44	0.80	10/10	−0.19	0.41	5/5
1000	−8.17	0.90	10/10	−0.32	0.38	5/5

Panel B: validated ensemble with DRUP		
	Coat	KuaiRec
Test nDCG (mean ± sd)		
Ensemble s	0.6031 ± 0.0164 (Coat)	0.6376 ± 0.0022 (KuaiRec)
A (final), $w = 0$	0.6015 ± 0.0162	0.6285 ± 0.0022
DRUP ($w = 1$)	0.5467 ± 0.0235	0.6378 ± 0.0019
Fixed blend, $w = 0.5$	0.5889 ± 0.0134	0.6361 ± 0.0023
Chosen w per split	0.0 (9 splits), 0.3 (1)	0.8 (1 split), 1.0 (4)
s vs A: diff [CI] / p_{Holm}	+1.79 [2.39] 0.1421	+8.76 [2.01] < 0.001
s vs DRUP: diff [CI] / p_{Holm}	+59.03 [24.57] < 0.001	−0.18 [0.19] 0.0664
s vs iALS (Coat) / DR-JL (KuaiRec)	+21.81 [20.25] 0.0823	+4.20 [2.19] 0.0005